

Aula Detalhada - Somadores

Tema do dia: half-adder, full-adder, ripple-carry adder, carry-in, carry-out, diferença entre carry e overflow e ideia de carry look-ahead **Aula na sequência:** 19 **Objetivo:** entender como a soma binária é implementada em hardware, desde uma soma de 1 bit até somadores de vários bits, e saber interpretar carry e overflow em questões de prova.

1. Onde Esta Aula Entra No Estudo?

Nas primeiras aulas, você estudou soma binária como operação numérica.

Agora a pergunta muda:

como o circuito faz essa soma?

Para responder isso, vamos sair da conta escrita no papel e entrar nos blocos lógicos.

O caminho da aula é:

soma de 1 bit sem carry de entrada
-> half-adder
-> soma de 1 bit com carry de entrada
-> full-adder
-> vários full-adders em sequência
-> ripple-carry adder
-> carry, overflow e atraso
-> ideia de carry look-ahead

Essa aula conversa diretamente com:

Aula 1 -> soma binária, sinal e complemento de 2
Aula 3 -> portas XOR, AND e OR
Aula 9 -> síntese lógica
Aula 15 -> atraso de propagação e caminho crítico

2. Retomada Curta: Soma Binária

Em binário, a soma de bits segue estas regras:

Soma	Resultado escrito	Bit de soma	Carry
0 + 0	0	0	0
0 + 1	1	1	0
1 + 0	1	1	0
1 + 1	10	0	1

O caso especial é:

$$1 + 1 = 10$$

Isso significa:

bit de soma = 0
vai 1 para a próxima coluna

Esse "vai 1" é o carry.

3. Half-Adder

3.1 O que é

O half-adder é um somador de 1 bit com duas entradas:

A
B

Ele gera duas saídas:

S -> bit de soma
Cout -> carry de saída

Ele é chamado de "half" porque não recebe carry de entrada.

Então ele resolve apenas:

$$A + B$$

3.2 Tabela-verdade do half-adder

A	B	Soma decimal	S	Cout
0	0	0	0	0
0	1	1	1	0
1	0	1	1	0
1	1	2	0	1

Observe:

S vale 1 quando A e B são diferentes.
Cout vale 1 quando A e B são 1 ao mesmo tempo.

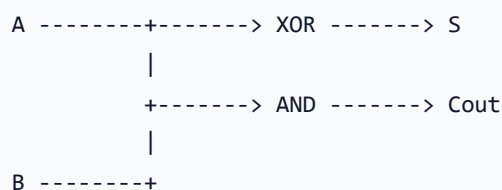
Logo:

$S = A \text{ XOR } B$
 $\text{Cout} = A.B$

Em notação de portas:

$S = A \oplus B$
 $\text{Cout} = A.B$

3.3 Circuito mental do half-adder



O mesmo par de entradas alimenta uma XOR e uma AND.

A XOR gera a soma.

A AND gera o carry.

4. Full-Adder

4.1 Por que o half-adder não basta?

Quando somamos números com mais de 1 bit, cada coluna pode receber um carry vindo da coluna anterior.

Exemplo:

```
1  <- carry vindo da coluna da direita
011
+ 001
----
100
```

Na coluna do meio, não somamos apenas $A + B$.

Somamos:

$$A + B + C_{in}$$

C_{in} é o carry de entrada.

Por isso precisamos do full-adder.

4.2 O que é

O full-adder é um somador de 1 bit com três entradas:

A
B
 C_{in}

E duas saídas:

S
Cout

Ele resolve:

$$A + B + C_{in}$$

4.3 Tabela-verdade do full-adder

A	B	Cin	Soma decimal	S	Cout
0	0	0	0	0	0
0	0	1	1	1	0
0	1	0	1	1	0
0	1	1	2	0	1
1	0	0	1	1	0
1	0	1	2	0	1
1	1	0	2	0	1
1	1	1	3	1	1

Leia a tabela assim:

S é o bit menos significativo da soma.
Cout é o bit que passa para a próxima coluna.

Exemplo:

A = 1, B = 1, Cin = 1

1 + 1 + 1 = 3 decimal

3 em binário = 11

S = 1

Cout = 1

4.4 Expressão da soma

A saída de soma vale 1 quando há quantidade ímpar de entradas iguais a 1.

Isso é XOR de três entradas:

$S = A \text{ XOR } B \text{ XOR } \text{Cin}$

Ou:

$S = A \oplus B \oplus \text{Cin}$

4.5 Expressão do carry de saída

O carry de saída vale 1 quando pelo menos duas entradas são 1.

Então:

$$\text{Cout} = A.B + A.\text{Cin} + B.\text{Cin}$$

Essa é a forma "maioria de 3":

se pelo menos dois entre A, B e Cin forem 1, gera carry

Outra forma muito usada:

$$\text{Cout} = A.B + \text{Cin}.(A \text{ XOR } B)$$

As duas são equivalentes.

5. Construindo Um Full-Adder Com Dois Half-Adders

Um jeito clássico de montar um full-adder é usar dois half-adders.

Passo 1:

$$A + B$$

gera:

$$\begin{aligned} S1 &= A \text{ XOR } B \\ C1 &= A.B \end{aligned}$$

Passo 2:

$$S1 + \text{Cin}$$

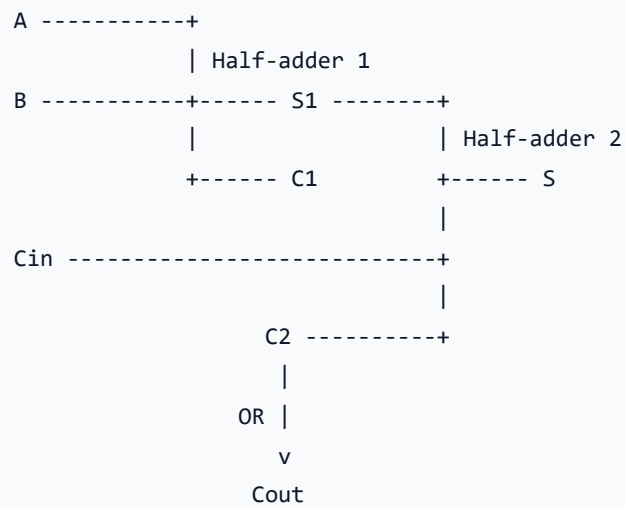
gera:

$$\begin{aligned} S &= S1 \text{ XOR } \text{Cin} \\ C2 &= S1.\text{Cin} \end{aligned}$$

O carry final é:

$$\text{Cout} = \text{C1} + \text{C2}$$

Diagrama:



Resumo:

$$\begin{aligned}
 S1 &= A \text{ XOR } B \\
 S &= S1 \text{ XOR } \text{Cin} \\
 \text{Cout} &= A.B + \text{Cin}.(A \text{ XOR } B)
 \end{aligned}$$

6. Ripple-Carry Adder

6.1 Ideia principal

Para somar números de vários bits, usamos vários full-adders em cascata.

Cada full-adder soma uma coluna.

O carry de saída de uma coluna vira o carry de entrada da próxima.

Exemplo de 4 bits:

$$\begin{array}{r}
 \text{A3} \text{ A2} \text{ A1} \text{ A0} \\
 + \text{B3} \text{ B2} \text{ B1} \text{ B0} \\
 \hline
 \text{C4} \text{ S3} \text{ S2} \text{ S1} \text{ S0}
 \end{array}$$

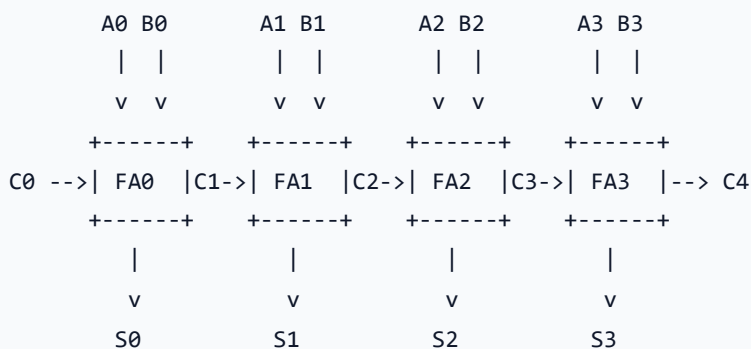
Os carries internos são:

C0 -> carry de entrada inicial
 C1 -> carry depois do bit 0
 C2 -> carry depois do bit 1
 C3 -> carry depois do bit 2
 C4 -> carry final

Normalmente, em uma soma comum:

C0 = 0

6.2 Diagrama de um somador de 4 bits



FA significa full-adder.

O nome ripple-carry vem da ideia de que o carry "ondula" de uma etapa para outra:

C0 -> C1 -> C2 -> C3 -> C4

6.3 Por que isso importa para temporização?

O ripple-carry é simples, mas pode ser lento.

Para saber o último bit da soma, o circuito pode precisar esperar o carry passar por vários full-adders.

Em um somador de 4 bits:

C1 depende de FA0
 C2 depende de FA1
 C3 depende de FA2
 C4 depende de FA3

Em um somador de 32 bits, esse caminho pode atravessar 32 estágios.

Conexão com a Aula 15:

mais estágios de portas -> maior atraso combinacional -> menor frequência máxima

7. Exemplo Completo De Ripple-Carry

Some:

```
0110
+ 0101
```

Vamos separar por bits:

```
A = 0110
B = 0101

A3=0 A2=1 A1=1 A0=0
B3=0 B2=1 B1=0 B0=1
```

Começamos com:

```
C0 = 0
```

Tabela por coluna:

Bit	A	B	Cin	Soma A+B+Cin	S	Cout
0	0	1	0	1	1	0
1	1	0	0	1	1	0
2	1	1	0	2	0	1
3	0	0	1	1	1	0

Resultado:

```
S3 S2 S1 S0 = 1011
C4 = 0
```

Então:

```
0110 + 0101 = 1011
```

Em decimal sem sinal:

```
6 + 5 = 11
```

E **1011** sem sinal é 11.

8. Carry-In E Carry-Out

8.1 Carry-in

Carry-in é o carry que entra em uma coluna.

Em um somador de vários bits:

```
Cin do bit 0 = C0  
Cin do bit 1 = C1  
Cin do bit 2 = C2  
Cin do bit 3 = C3
```

Na soma normal:

```
C0 = 0
```

Mas em outros circuitos, **C0** pode ser usado para fazer operações úteis.

Exemplo:

```
A + B + 1
```

Isso aparece em subtração por complemento de 2:

```
A - B = A + B' + 1
```

Esse **+1** pode entrar como carry-in inicial.

8.2 Carry-out

Carry-out é o carry que sai de uma coluna.

No último bit, o carry-out final pode indicar estouro em soma sem sinal.

Exemplo de 4 bits sem sinal:

```
  1111  
+ 0001  
-----  
1 0000
```

O resultado armazenado em 4 bits é:

```
0000
```

Mas o carry-out final é:

`C4 = 1`

Para números sem sinal, isso significa que o resultado passou da capacidade de 4 bits.

9. Carry E Overflow Não São Sempre A Mesma Coisa

Essa parte é muito importante para prova.

9.1 Em soma sem sinal

Para números sem sinal:

`carry-out final = estouro da capacidade`

Exemplo com 4 bits sem sinal:

`1111 + 0001 = 1 0000`

Como 4 bits só armazenam:

`0000 até 1111`
`0 até 15`

`15 + 1 = 16` não cabe em 4 bits.

Então:

`Cout final = 1`
`houve overflow sem sinal`

9.2 Em complemento de 2

Para números com sinal em complemento de 2, o carry-out final não decide sozinho se houve overflow.

Regra prática:

`somou dois positivos e deu negativo -> overflow`
`somou dois negativos e deu positivo -> overflow`
`somou sinais diferentes -> não há overflow`

Outra regra equivalente:

overflow = carry que entra no bit de sinal XOR carry que sai do bit de sinal

Em 4 bits, o bit de sinal é o bit 3.

Então:

Overflow = C3 XOR C4

9.3 Exemplo: carry sem overflow em complemento de 2

Some em 4 bits:

```

  1101
+ 1110
-----
1 1011

```

O resultado armazenado em 4 bits é:

1011

Interpretação em complemento de 2:

```

1101 = -3
1110 = -2
1011 = -5

```

A conta real é:

-3 + -2 = -5

O resultado cabe em 4 bits, pois o intervalo é:

-8 até +7

Então:

houve carry-out final
mas não houve overflow com sinal

Esse exemplo é o tipo de pegadinha que diferencia carry de overflow.

9.4 Exemplo: overflow sem carry-out final

Some em 4 bits:

```
0111
+ 0001
-----
1000
```

Interpretação em complemento de 2:

```
0111 = +7
0001 = +1
1000 = -8
```

Mas:

```
7 + 1 = 8
```

O valor **+8** não cabe em 4 bits com sinal, pois o intervalo é:

```
-8 até +7
```

Então:

```
houve overflow com sinal
mesmo sem carry-out final
```

10. Carry Propagate E Carry Generate

Esta parte prepara a ideia de carry look-ahead.

Para cada bit, podemos dizer que ele:

```
gera carry
propaga carry
```

10.1 Generate

Um bit gera carry quando **A** e **B** são 1.

```
G = A.B
```

Se **A=1** e **B=1**, haverá carry independente do carry de entrada.

Exemplo:

$1 + 1 + \text{Cin}$

Sempre gera carry.

10.2 Propagate

Um bit propaga carry quando **A** e **B** são diferentes.

 $P = A \text{ XOR } B$

Se **A** e **B** são diferentes, o carry de entrada passa para a saída.

Exemplo:

 $A=1, B=0$

Se **Cin=0**:

$$1 + 0 + 0 = 1$$
$$\text{Cout} = 0$$

Se **Cin=1**:

$$1 + 0 + 1 = 10$$
$$\text{Cout} = 1$$

Então o carry de entrada foi propagado.

10.3 Fórmulas úteis

Para o bit **i**:

$$P_i = A_i \text{ XOR } B_i$$
$$G_i = A_i \cdot B_i$$
$$S_i = P_i \text{ XOR } C_i$$
$$C_{(i+1)} = G_i + P_i \cdot C_i$$

Tradução:

o próximo carry existe se o bit gerou carry
ou se o bit propagou o carry que já vinha entrando

11. Carry Look-Ahead Em Nível Conceitual

11.1 Problema do ripple-carry

No ripple-carry, o carry precisa atravessar estágio por estágio.

Para descobrir C_4 , o circuito espera:

```
C1
depois C2
depois C3
depois C4
```

Isso aumenta o atraso.

11.2 Ideia do carry look-ahead

O carry look-ahead tenta calcular os carries mais rapidamente usando as fórmulas de generate e propagate.

Em vez de esperar o carry "andar" por todos os full-adders, o circuito calcula expressões diretas.

Para 4 bits:

```
C1 = G0 + P0.C0
C2 = G1 + P1.C1
C3 = G2 + P2.C2
C4 = G3 + P3.C3
```

Substituindo para não depender em cascata:

```
C1 = G0 + P0.C0

C2 = G1 + P1.G0 + P1.P0.C0

C3 = G2 + P2.G1 + P2.P1.G0 + P2.P1.P0.C0

C4 = G3 + P3.G2 + P3.P2.G1 + P3.P2.P1.G0 + P3.P2.P1.P0.C0
```

Não precisa decorar essas expressões grandes para esta prova.

O que importa é entender:

```
ripple-carry -> simples, mas o carry passa em cadeia
carry look-ahead -> mais lógica, mas calcula carries mais rápido
```

11.3 Como a prova costuma cobrar

Ela pode perguntar:

qual somador é mais simples?
qual somador tende a ser mais lento?
por que ripple-carry tem atraso maior?
para que servem generate e propagate?

Respostas:

ripple-carry é simples
ripple-carry é mais lento em muitos bits
o atraso vem da propagação sequencial do carry
generate cria carry; propagate deixa o carry passar

12. Como Questões Costumam Cobrar

12.1 Identificar portas do half-adder

Pergunta típica:

Quais portas geram S e Cout em um half-adder?

Resposta:

$S = \text{XOR}$
 $\text{Cout} = \text{AND}$

12.2 Identificar expressões do full-adder

Pergunta típica:

Qual expressão representa o carry de saída de um full-adder?

Resposta:

$\text{Cout} = A.B + A.C_{in} + B.C_{in}$

ou:

$\text{Cout} = A.B + C_{in}.(A \text{ XOR } B)$

12.3 Rastrear ripple-carry

Pergunta típica:

Some dois números de 4 bits e indique S e Cout.

Faça coluna por coluna:

bit 0 -> gera C1
bit 1 -> usa C1 e gera C2
bit 2 -> usa C2 e gera C3
bit 3 -> usa C3 e gera C4

12.4 Diferenciar carry e overflow

Pergunta típica:

Em complemento de 2, houve overflow?

Não responda olhando só o carry final.

Use:

dois positivos deram negativo?
dois negativos deram positivo?
 $C3 \text{ XOR } C4 = 1$?

13. Erros Comuns

13.1 Achar que todo carry é overflow

Isso só funciona diretamente para soma sem sinal.

Em complemento de 2:

carry-out final pode existir sem overflow
overflow pode existir sem carry-out final

13.2 Esquecer o carry de entrada no full-adder

Se a questão fala de full-adder, sempre considere:

A
B
Cin

Não é só $A+B$.

13.3 Ler o resultado de 4 bits como se tivesse 5 bits

Se o circuito é de 4 bits, o registrador guarda apenas:

S3 S2 S1 S0

O **Cout** pode ser uma saída separada, mas não faz parte automaticamente do número armazenado em 4 bits.

13.4 Confundir bit menos significativo e bit mais significativo

Em:

A3 A2 A1 A0

O bit **A0** é o menos significativo.

Ele é somado primeiro e gera **C1**.

O bit **A3** é o mais significativo.

Em complemento de 2, **A3** também é o bit de sinal.

14. Exemplos Resolvidos

14.1 Half-adder

Entradas:

A = 1
B = 1

Saídas:

$S = A \text{ XOR } B = 1 \text{ XOR } 1 = 0$
 $\text{Cout} = A \cdot B = 1 \cdot 1 = 1$

Resultado:

$$1 + 1 = 10$$

14.2 Full-adder

Entradas:

$$\begin{aligned} A &= 1 \\ B &= 0 \\ C_{in} &= 1 \end{aligned}$$

Soma decimal:

$$1 + 0 + 1 = 2$$

Em binário:

$$2 = 10$$

Logo:

$$\begin{aligned} S &= 0 \\ C_{out} &= 1 \end{aligned}$$

Pelas fórmulas:

$$\begin{aligned} S &= A \text{ XOR } B \text{ XOR } C_{in} \\ S &= 1 \text{ XOR } 0 \text{ XOR } 1 \\ S &= 0 \\ \\ C_{out} &= A.B + A.C_{in} + B.C_{in} \\ C_{out} &= 1.0 + 1.1 + 0.1 \\ C_{out} &= 0 + 1 + 0 \\ C_{out} &= 1 \end{aligned}$$

14.3 Ripple-carry com carry final

Some:

$$\begin{array}{r} 1111 \\ + 0001 \\ \hline \end{array}$$

Tabela:

Bit	A	B	Cin	S	Cout
0	1	1	0	0	1
1	1	0	1	0	1
2	1	0	1	0	1
3	1	0	1	0	1

Resultado armazenado em 4 bits:

0000

Carry final:

C4 = 1

Sem sinal:

15 + 1 = 16

Não cabe em 4 bits, então houve overflow sem sinal.

Com sinal:

1111 = -1
0001 = +1
0000 = 0

Sinais diferentes:

não há overflow com sinal

15. Exercícios

15.1 Conceituais

1. Quais são as entradas e saídas de um half-adder?
2. Quais são as entradas e saídas de um full-adder?
3. Por que um half-adder não é suficiente para somar todos os bits de um número de vários bits?
4. Em uma soma sem sinal, o que indica Cout=1 no último bit?
5. Em complemento de 2, por que não basta olhar apenas o Cout final para decidir overflow?

15.2 Aplicação direta

6. Para um half-adder com $A=1$ e $B=0$, determine S e $Cout$.
7. Para um full-adder com $A=1$, $B=1$ e $Cin=0$, determine S e $Cout$.
8. Para um full-adder com $A=0$, $B=1$ e $Cin=1$, determine S e $Cout$.
9. Em um full-adder, calcule $Cout$ usando $Cout = A.B + A.Cin + B.Cin$ para $A=1$, $B=0$, $Cin=1$.
10. Para $A=1$, $B=0$, determine P e G .

15.3 Ripple-carry e overflow

11. Some $0101 + 0011$ em 4 bits. Informe o resultado e o carry final.
12. Some $1110 + 0011$ em 4 bits. Informe o resultado e o carry final.
13. Em 4 bits sem sinal, a soma $1110 + 0011$ teve overflow? Explique.
14. Em 4 bits com sinal em complemento de 2, a soma $1110 + 0011$ teve overflow? Explique.
15. Em 4 bits com sinal em complemento de 2, a soma $0111 + 0001$ teve overflow? Explique.
16. Em 4 bits com sinal em complemento de 2, a soma $1101 + 1110$ teve overflow? Explique.

15.4 Questões mais parecidas com prova

17. Um somador ripple-carry de 8 bits é feito com full-adders. Qual é o principal motivo para ele ser mais lento que um somador carry look-ahead?
18. Em um full-adder, qual alternativa representa corretamente a saída de soma?

- A) $A.B + Cin$
- B) $A \text{ XOR } B \text{ XOR } Cin$
- C) $A + B + Cin$
- D) $A.B.Cin$

19. Em um full-adder, qual alternativa representa corretamente a saída de carry?

- A) $A \text{ XOR } B \text{ XOR } Cin$
- B) $A.B + A.Cin + B.Cin$
- C) $A + B + Cin$
- D) $A'.B'.Cin'$

20. Em um somador de 4 bits, $C3=1$ e $C4=0$. Houve overflow com sinal? Explique usando a regra dos carries.

16. Gabarito

1. Entradas: A e B . Saídas: S e $Cout$.

2. Entradas: A , B e C_{in} . Saídas: S e C_{out} .
3. Porque, em somas de vários bits, uma coluna pode receber carry da coluna anterior. O half-adder não tem entrada C_{in} .
4. Indica que o resultado ultrapassou a capacidade de bits disponíveis. Em soma sem sinal, isso é overflow da representação.
5. Porque, em complemento de 2, overflow depende do sinal dos operandos e do resultado, ou de $C_3 \text{ XOR } C_4$, não apenas de C_{out} .
6. $1 + 0 = 1$, então $S=1$ e $C_{out}=0$.
7. $1 + 1 + 0 = 2 = 10$, então $S=0$ e $C_{out}=1$.
8. $0 + 1 + 1 = 2 = 10$, então $S=0$ e $C_{out}=1$.
9. $C_{out} = 1.0 + 1.1 + 0.1 = 0 + 1 + 0 = 1$.
10. $P = A \text{ XOR } B = 1$; $G = A.B = 0$.
11. $0101 + 0011 = 1000$, com carry final 0 .
12. $1110 + 0011 = 1\ 0001$. Resultado armazenado em 4 bits: 0001 . Carry final: 1 .
13. Sim. Sem sinal, $1110=14$ e $0011=3$; $14+3=17$, que não cabe em 4 bits sem sinal.
14. Não. Em complemento de 2, $1110=-2$ e $0011=+3$; sinais diferentes não geram overflow. Resultado $0001=+1$.
15. Sim. $0111=+7$ e $0001=+1$; o resultado armazenado é 1000 , que representa -8 . Dois positivos deram negativo.
16. Não. $1101=-3$ e $1110=-2$; resultado $1011=-5$, que cabe em 4 bits. Há carry final, mas não há overflow com sinal.
17. Porque o carry precisa se propagar de um full-adder para o próximo. O atraso cresce com a quantidade de bits.
18. Alternativa B.
19. Alternativa B.
20. Sim. Pela regra $\text{overflow} = C_3 \text{ XOR } C_4$: $1 \text{ XOR } 0 = 1$. Portanto, houve overflow com sinal.

17. O Que Memorizar

Half-adder:

$$S = A \text{ XOR } B$$

$$C_{out} = A.B$$

Full-adder:

$$S = A \text{ XOR } B \text{ XOR } C_{in}$$

$$C_{out} = A.B + A.C_{in} + B.C_{in}$$

Ripple-carry:

o Cout de um bit vira o Cin do próximo bit

Soma sem sinal:

Cout final indica overflow da capacidade

Complemento de 2:

overflow ocorre quando dois positivos dão negativo

ou dois negativos dão positivo

Outra regra de overflow com sinal:

overflow = carry que entra no bit de sinal XOR carry que sai do bit de sinal

Carry look-ahead:

usa propagate e generate para calcular carries mais rápido

18. Plano De Estudo Para Esta Aula

Sugestão de estudo:

1. Ler seções 2 a 5 para fixar half-adder e full-adder.
2. Copiar a tabela do full-adder uma vez.
3. Refazer o exemplo de ripple-carry da seção 7.
4. Resolver exercícios 6 a 16 sem consultar.
5. Corrigir carry vs overflow com atenção.
6. Fazer 5 minutos de Anki sobre:
 - half-adder
 - full-adder
 - carry
 - overflow
 - ripple-carry

Se for fazer poucos exercícios, priorize:

11, 12, 13, 14, 15 e 16

Eles treinam exatamente a parte que mais confunde em prova.

19. Conexão Com A Próxima Aula

Esta aula funciona como reforço operacional para a aula de subtrator, comparador, shifter e ULA.

Depois dela, a ideia que precisa ficar sólida é:

o mesmo somador pode ser reaproveitado para soma e subtração

Isso acontece porque:

$$A - B = A + B' + 1$$

O somador desta aula também será importante nas revisões de ULA, datapath e arquitetura.